

Pipeline (computing)

In computing, a **pipeline**, also known as a **data pipeline**, is a set of data processing elements connected in series, where the output of one element is the input of the next one. The elements of a pipeline are often executed in parallel or in time-sliced fashion. Some amount of buffer storage is often inserted between elements.

Concept and motivation

Pipelining is a commonly used concept in everyday life. For example, in the assembly line of a car factory, each specific task—such as installing the engine, installing the hood, and installing the wheels—is often done by a separate work station. The stations carry out their tasks in parallel, each on a different car. Once a car has had one task performed, it moves to the next station. Variations in the time needed to complete the tasks can be accommodated by "buffering" (holding one or more cars in a space between the stations) and/or by "stalling" (temporarily halting the upstream stations), until the next station becomes available.

Suppose that assembling one car requires three tasks that take 20, 10, and 15 minutes, respectively. Then, if all three tasks were performed by a single station, the factory would output one car every 45 minutes. By using a pipeline of three stations, the factory would output the first car in 45 minutes, and then a new one every 20 minutes.

As this example shows, pipelining does not decrease the latency, that is, the total time for one item to go through the whole system. It does however increase the system's throughput, that is, the rate at which new items are processed after the first one.

In computing

In computing, a pipeline or data pipeline^[1] is a set of data processing elements connected in series, where the output of one element is the input of the next one. The elements of a pipeline are often executed in parallel or in time-sliced fashion. Some amount of buffer storage is often inserted between elements.

Computer-related pipelines include:

- Instruction pipelines, such as the classic RISC pipeline, which are used in central processing units (CPUs) and other microprocessors to allow overlapping execution of multiple instructions with the same circuitry. The circuitry is usually divided up into stages and each stage processes a specific part of one instruction at a time, passing the partial results to the next stage. Examples of stages are instruction decode, arithmetic/logic and register fetch. They are related to the technologies of superscalar execution, operand forwarding, speculative execution and out-of-order execution.
- Graphics pipelines, found in most graphics processing units (GPUs), which consist of multiple arithmetic units, or complete CPUs, that implement the various stages of common rendering operations (perspective projection, window clipping, color and light calculation, rendering, etc.).

- Software pipelines, which consist of a sequence of computing processes (commands, program runs, tasks, threads, procedures, etc.), conceptually executed in parallel, with the output stream of one process being automatically fed as the input stream of the next one. The Unix system call pipe is a classic example of this concept.
- HTTP pipelining, the technique of issuing multiple HTTP requests through the same TCP connection, without waiting for the previous one to finish before issuing a new one.

If a pipeline consists of idempotent steps, these can be rerun without altering the outcome beyond the initial run. This is advantageous in terms of reliability and maintenance.

Design considerations

Balancing the stages

Since the throughput of a pipeline cannot be better than that of its slowest element, the designer should try to divide the work and resources among the stages so that they all take the same time to complete their tasks. In the car assembly example above, if the three tasks took 15 minutes each, instead of 20, 10, and 15 minutes, the latency would still be 45 minutes, but a new car would then be finished every 15 minutes, instead of 20.

Buffering

Under ideal circumstances, if all processing elements are synchronized and take the same amount of time to process, then each item can be received by each element just as it is released by the previous one, in a single clock cycle. That way, the items will flow through the pipeline at a constant speed, like waves in a water channel. In such "wave pipelines",^[2] no synchronization or buffering is needed between the stages, besides the storage needed for the data items.

More generally, buffering between the pipeline stages is necessary when the processing times are irregular, or when items may be created or destroyed along the pipeline. For example, in a graphics pipeline that processes triangles to be rendered on the screen, an element that checks the visibility of each triangle may discard the triangle if it is invisible, or may output two or more triangular pieces of the element if they are partly hidden. Buffering is also needed to accommodate irregularities in the rates at which the application feeds items to the first stage and consumes the output of the last one.

The buffer between two stages may be simply a hardware register with suitable synchronization and signalling logic between the two stages. When a stage A stores a data item in the register, it sends a "data available" signal to the next stage B. Once B has used that data, it responds with a "data received" signal to A. Stage A halts, waiting for this signal, before storing the next data item into the register. Stage B halts, waiting for the "data available" signal, if it is ready to process the next item but stage A has not provided it yet.

If the processing times of an element are variable, the whole pipeline may often have to stop, waiting for that element and all the previous ones to consume the items in their input buffers. The frequency of such pipeline stalls can be reduced by providing space for more than one item in the input buffer of that stage. Such a multiple-item buffer is usually implemented as a first-in, first-out queue. The upstream stage may

still have to be halted when the queue gets full, but the frequency of those events will decrease as more buffer slots are provided. Queueing theory can tell the number of buffer slots needed, depending on the variability of the processing times and on the desired performance.

Nonlinear pipelines

If some stage takes (or may take) much longer than the others, and cannot be sped up, the designer can provide two or more processing elements to carry out that task in parallel, with a single input buffer and a single output buffer. As each element finishes processing its current data item, it delivers it to the common output buffer, and takes the next data item from the common input buffer. This concept of "non-linear" or "dynamic" pipeline is exemplified by shops or banks that have two or more cashiers serving clients from a single waiting queue.

Dependencies between items

In some applications, the processing of an item Y by a stage A may depend on the results or effect of processing a previous item X by some later stage B of the pipeline. In that case, stage A cannot correctly process item Y until item X has cleared stage B.

This situation occurs very often in instruction pipelines. For example, suppose that Y is an arithmetic instruction that reads the contents of a register that was supposed to have been modified by an earlier instruction X. Let A be the stage that fetches the instruction operands, and B be the stage that writes the result to the specified register. If stage A tries to process instruction Y before instruction X reaches stage B, the register may still contain the old value, and the effect of Y would be incorrect.

In order to handle such conflicts correctly, the pipeline must be provided with extra circuitry or logic that detects them and takes the appropriate action. Strategies for doing so include:

- **Stalling:** Every affected stage, such as A, is halted until the dependency is resolved—that is, until the required information is available and/or the required state has been achieved.
- **Reordering items:** Instead of stalling, stage A may put item Y aside and look for any subsequent item Z in its input stream that does not have any dependencies pending with any earlier item. In instruction pipelines, this technique is called out-of-order execution.
- **Guess and backtrack:** One important example of item-to-item dependency is the handling of a conditional branch instruction X by an instruction pipeline. The first stage A of the pipeline, that fetches the next instruction Y to be executed, cannot perform its task until X has fetched its operand and determined whether the branch is to be taken or not. That may take many clock cycles, since the operand of X may in turn depend on previous instructions that fetch data from main memory.

Rather than halt while waiting for X to be finished, stage A may guess whether the branch will be taken or not, and fetch the next instruction Y based on that guess. If the guess later turns out to be incorrect (hopefully rarely), the system would have to backtrack and resume with the correct choice. Namely, all the changes that were made to the machine's state by stage A and subsequent stages based on that guess would have to be undone, the instructions following X already in the pipeline would have to be flushed, and stage A would have to restart with the correct instruction pointer. This branch prediction strategy is a special case of speculative execution.

Typical software implementations

To be effectively implemented, data pipelines need a CPU scheduling strategy to dispatch work to the available CPU cores, and the usage of data structures on which the pipeline stages will operate on. For example, UNIX derivatives may pipeline commands connecting various processes' standard IO, using the pipes implemented by the operating system. Some operating systems may provide UNIX-like syntax to string several program runs in a pipeline, but implement the latter as simple serial execution, rather than true pipelining—namely, by waiting for each program to finish before starting the next one.

Lower level approaches may rely on the threads provided by the operating system to schedule work on the stages: both thread pool-based implementations or on a one-thread-per-stage are viable, and exist.^[3]

Other strategies relying on cooperative multitasking exist, that do not need multiple threads of execution and hence additional CPU cores, such as using a round-robin scheduler with a coroutine-based framework. In this context, each stage may be instantiated with its own coroutine, yielding control back to the scheduler after finishing its round task. This approach may need careful control over the process' stages to avoid them abuse their time slice.

Costs and drawbacks

A pipelined system typically requires more resources (circuit elements, processing units, computer memory, etc.) than one that executes one batch at a time, because its stages cannot share those resources, and because buffering and additional synchronization logic may be needed between the elements.

Moreover, the transfer of items between separate processing elements may increase the latency, especially for long pipelines.

The additional complexity cost of pipelining may be considerable if there are dependencies between the processing of different items, especially if a guess-and-backtrack strategy is used to handle them. Indeed, the cost of implementing that strategy for complex instruction sets has motivated some radical proposals to simplify computer architecture, such as RISC and VLIW. Compilers also have been burdened with the task of rearranging the machine instructions so as to improve the performance of instruction pipelines.

New technologies

In recent years the demands on applications and their underlying hardware have been significant. For example, building pipelines with single node applications that trawl through the data row by row is no longer feasible with the volume and variety of big data. However, with the advent of data analytics engines such as Hadoop, or more recently Apache Spark, it's been possible to distribute large datasets across multiple processing nodes, allowing applications to reach heights of efficiency several hundred times greater than was thought possible before. The effect of this today is that even a mid-level PC using distributed processing in this fashion can handle the building and running of big data pipelines.^[4]

See also

- [Dataflow](#)
- [Throughput](#)
- [Parallelism](#)
- [Instruction pipeline](#)
 - [Classic RISC pipeline](#)
- [Graphics pipeline](#)
- [Pipeline \(software\)](#)
 - [Pipeline \(Unix\)](#)
 - [Hartmann pipeline](#) for VM
 - [BatchPipes](#) for MVS
- [Geometry pipelines](#)
- [XML pipeline](#)
- [Staged event-driven architecture](#)

References

1. Data Pipeline Development (<https://www.dativa.com/data-pipelines/>) Archived (<https://web.archive.org/web/20180524151334/https://www.dativa.com/data-pipelines/>) 2018-05-24 at the Wayback Machine Published by Dativa, retrieved 24 May, 2018
2. O. Hauck; Sorin A. Huss; M. Garg (1999). "Two-phase asynchronous wave-pipelines and their application to a 2D-DCT". *Proceedings. Fifth International Symposium on Advanced Research in Asynchronous Circuits and Systems*. pp. 219–228. doi:10.1109/ASYNC.1999.761536 (<https://doi.org/10.1109%2FASYNC.1999.761536>). ISBN 0-7695-0031-5. S2CID 206515615 (<https://api.semanticscholar.org/CorpusID:206515615>).
3. "MTDP" (<https://github.com/dteod/mtdp.git/>). *GitHub*. September 2022.
4. What is a Data Pipeline? (<https://www.datapipelines.com/blog/what-is-a-data-pipeline/>) Published by Data Pipelines, retrieved 11 March 2021

Bibliography

- Perez Garcia, Pablo (2018). *Pipeline DSL a dsl to create a CI/CD pipeline for your projects* (https://github.com/politrons/Pipeline_DSL/). Addison-Wesley. ISBN 978-0-134-69147-3.
- For a standard discussion on pipelining in parallel computing see Quinn, Michael J. (2004). *Parallel Programming in C with MPI and openMP* (<https://archive.org/details/parallelprogramm0000quin>). Dubuque, Iowa: McGraw-Hill Professional. ISBN 0072822562.
- Pogonyi, Roland (February 2021). "What is a Data Pipeline?" (<https://www.datapipelines.com/blog/what-is-a-data-pipeline/>). Retrieved March 11, 2021.

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Pipeline_\(computing\)&oldid=1349290888](https://en.wikipedia.org/w/index.php?title=Pipeline_(computing)&oldid=1349290888)"